Minimum spanning trees

Outline

- The minimum spanning tree problem
  - Applications
- Cuts and cycles
- The cut property and the cycle property
  - Optimality conditions
- The generic greedy algorithm
- Kruskal's algorithm
  - Implementation and running time
  - Kruskal's algorithm on the reference graph
- Prim's algorithm
  - Prim's algorithm on the reference graph
- Uniqueness of the minimum spanning tree
  - Weights with ties
- Typical mistakes

Learning objectives

At the end of this chapter you will be able to:

- state the minimum spanning tree problem and recognize it in applications;
- prove the cut property and the cycle property, and use them to certify that a spanning tree is optimal;
- run the algorithms of Kruskal and of Prim, prove that they are correct and analyse their running time;
- decide whether the minimum spanning tree is unique, and find the edges that belong to every optimal tree.

Prerequisites

- The chapter «Graphs»: paths and cycles, connected components, the characterizations of a tree and the fundamental cycles.
- The $O(\cdot)$ notation for the running time of an algorithm, and sorting m numbers in $O(m \log m)$ time.

The minimum spanning tree problem

Introduction: The minimum spanning tree problem

- We want to connect all the vertices of a weighted graph with a set of edges of minimum total weight
- The cheapest connected network is always a spanning tree: the problem asks for the best one
- In this section we state the problem, fix the weighted reference graph used in all the examples, and see where the problem appears in applications

Definition 1: minimum spanning tree problem

Let $G = (V, E)$ be a connected undirected graph with $n = |V|$ vertices and $m = |E|$ edges, and let every edge $e \in E$ have a ***weight*** *$w_e \in \mathbb{R}$. The* ***weight*** *of a set of edges $F \subseteq E$ is*

$$w(F) = \sum_{e \in F} w_e.$$

The ***minimum spanning tree problem*** *(MST) asks for a spanning tree $\tilde{T} = (V, \tilde{F})$ of G of minimum weight:*

$$w(\tilde{F}) \leq w(F) \qquad \text{for every spanning tree } (V, F) \text{ of } G.$$

Definition: minimum spanning tree problem (continued)

A spanning tree of minimum weight is a ***minimum spanning tree****. We write* w_{uv} *for the weight of the edge* $\{u, v\}$*, and we identify a spanning tree with its set of edges.*

First remarks

- A connected graph has a spanning tree, and it has finitely many of them: a minimum spanning tree always exists. Every spanning tree has $n - 1$ edges.
- The weights can be negative. Adding the same constant to every weight adds $(n - 1)$ times that constant to the weight of every spanning tree, so the minimum spanning trees do not change.
- A **maximum spanning tree** is a minimum spanning tree for the weights $-w_e$.

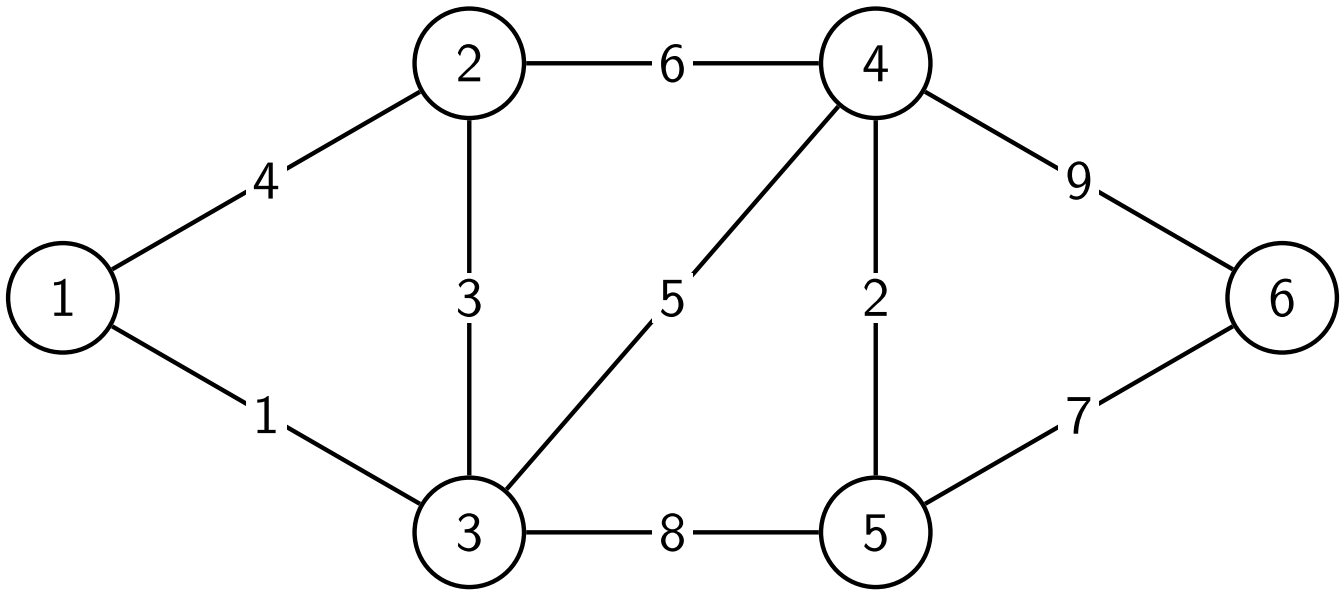
Before studying the problem, we fix one weighted graph:
all the examples of the chapter use it.

Example: the weighted reference graph (1/2)

- We take the underlying undirected graph of the reference graph of the chapter «Graphs», with $n = 6$ vertices and $m = 9$ edges
- The weights are

edge	$\{1,2\}$	$\{1,3\}$	$\{2,3\}$	$\{2,4\}$	$\{3,4\}$	$\{3,5\}$	$\{4,5\}$	$\{4,6\}$	$\{5,6\}$
w_e	4	1	3	6	5	8	2	9	7

Example: the weighted reference graph (2/2)



- We call this graph the **weighted reference graph**: all the examples of the chapter use it.
- The weights are the distinct integers $1, 2, \ldots, 9$.

Example: the minimum spanning tree of the reference graph (1/2)

- The weighted reference graph has 55 spanning trees
- Enumerating all of them, the minimum weight is 18, attained only by the tree

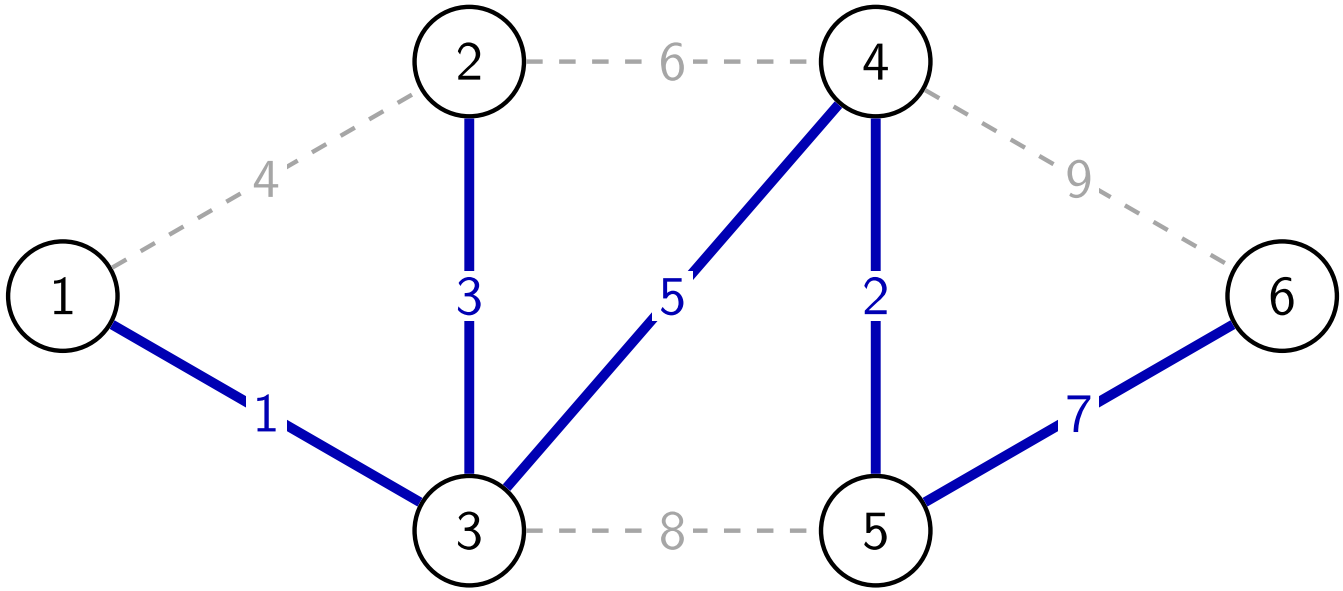
$$\tilde{F} = \{\{1,3\},\{2,3\},\{3,4\},\{4,5\},\{5,6\}\}$$

with

$$w(\tilde{F}) = 1 + 3 + 5 + 2 + 7 = 18$$

Example: the minimum spanning tree of the reference graph (2/2)

The tree $\tilde{F}$ is blue in the figure.



- The spanning tree $\{1,2\},\{1,3\},\{2,4\},\{3,5\},\{4,6\}$ found by the breadth-first search in the chapter «Graphs» weighs $4+1+6+8+9=28$
- A maximum spanning tree weighs 34

Why we need algorithms

- Enumerating the spanning trees is practical only on tiny graphs
- The reference graph, with 6 vertices and 9 edges, already has 55
- The algorithms of this chapter find a minimum spanning tree in polynomial time

Applications

- **Network design.** Vertices are sites (offices, houses, plants) and w_{uv} is the cost of a direct link (a cable, a pipe, a road); the cheapest network connecting all the sites is a minimum spanning tree.
- **Clustering.** Removing the $k-1$ heaviest edges of a minimum spanning tree splits the vertices into k groups of close points (single-linkage clustering).
- **Subproblem.** Minimum spanning trees give lower bounds for the travelling salesman problem and a starting point for heuristic algorithms for network design.

The minimum spanning tree in one sentence

A minimum spanning tree connects all the vertices of a connected weighted graph with $n - 1$ edges of minimum total weight; on the weighted reference graph it is unique and weighs 18.

Summary

The problem:

- Find a spanning tree of minimum total weight of a connected weighted graph
- It always exists and has $n - 1$ edges; negative weights and maximum spanning trees are allowed

The weighted reference graph:

- $n = 6$, $m = 9$, weights $1, 2, \ldots, 9$, 55 spanning trees
- Unique minimum spanning tree $\tilde{F}$, of weight 18

Cuts and cycles

Introduction: Cuts and cycles

- Cuts and cycles are the two tools behind every result of this chapter
- We define cuts and fundamental cuts, and prove that a cycle crosses a cut an even number of times
- Recall from the chapter «Graphs»: for a spanning tree T and an edge $e \notin T$, the **fundamental cycle** $C(T, e)$ is the unique cycle of $T \cup \{e\}$
- The exchange lemma tells how to move from one spanning tree to another

Definition 2: cut

*Let $G = (V, E)$ be an undirected graph and $S \subseteq V$ with $\emptyset \neq S \neq V$. The **cut** defined by S is the set of the edges with exactly one endpoint in S:*

$$\delta(S) = \{ \{u, v\} \in E : u \in S,\ v \notin S \}.$$

*An edge of $\delta(S)$ **crosses** the cut. For a single vertex, $\delta(\{v\}) = \delta(v)$ is the set of the edges incident to v.*

Cuts and connectivity

- A graph is connected if and only if every cut $\delta(S)$ is nonempty:
  - an empty cut separates S from $V \setminus S$;
  - a path from $u \in S$ to $v \notin S$ must use an edge of $\delta(S)$.

- In the weighted reference graph

$$\delta(\{1,2,3\}) = \{\{2,4\},\{3,4\},\{3,5\}\}$$

and

$$\delta(\{6\}) = \{\{4,6\},\{5,6\}\}$$

Lemma 1: a cycle crosses a cut an even number of times

Let C be a cycle of G and $\delta(S)$ a cut. Then $|C \cap \delta(S)|$ is even. In particular, if a cycle contains an edge of $\delta(S)$, it contains at least one other edge of $\delta(S)$.

Proof

- Let the cycle visit $v_0, v_1, \ldots, v_k = v_0$
- The edge $\{v_{t-1}, v_t\}$ crosses the cut if and only if v_{t-1} and v_t lie on different sides
- Walking along the cycle, the side changes at every crossing edge and only there
- The walk ends at v_0, on the side where it started, so the number of changes is even

□

Definition 3: fundamental cut

Let T be a spanning tree of a connected graph $G = (V, E)$ and $e = \{u, v\} \in T$. The graph $T \setminus \{e\}$ has two connected components; let S_e be the one that contains u. The ***fundamental cut*** *of e is $\delta(S_e)$.*

Why the fundamental cut is well defined

- $T \setminus \{e\}$ has $n-2$ edges and no cycle, so it has $n-(n-2)=2$ components (Corollary «edges and components»)
- v is not in S_e, otherwise the path from u to v in T would not be unique
- The only edge of T in the fundamental cut of e is e itself: every other edge of T lies inside one of the two components

With fundamental cycles and fundamental cuts
we can now exchange one edge of a spanning tree for another.

Lemma 2: exchange

Let T be a spanning tree of a connected graph $G = (V, E)$.

1. *If $e \notin T$ and f is an edge of the fundamental cycle $C(T, e)$, then $T \cup \{e\} \setminus \{f\}$ is a spanning tree.*
2. *If $f \in T$ and $e \notin T$ crosses the fundamental cut of f, then $f \in C(T, e)$, and $T \cup \{e\} \setminus \{f\}$ is a spanning tree.*

Proof (1/2)

Part 1.

- The rest of $C(T, e)$ joins the endpoints of f
- So removing f from the connected graph $T \cup \{e\}$ leaves it connected
- The result is connected with $n - 1$ edges: it is a spanning tree by the characterizations of a tree, (b)

Proof (2/2)

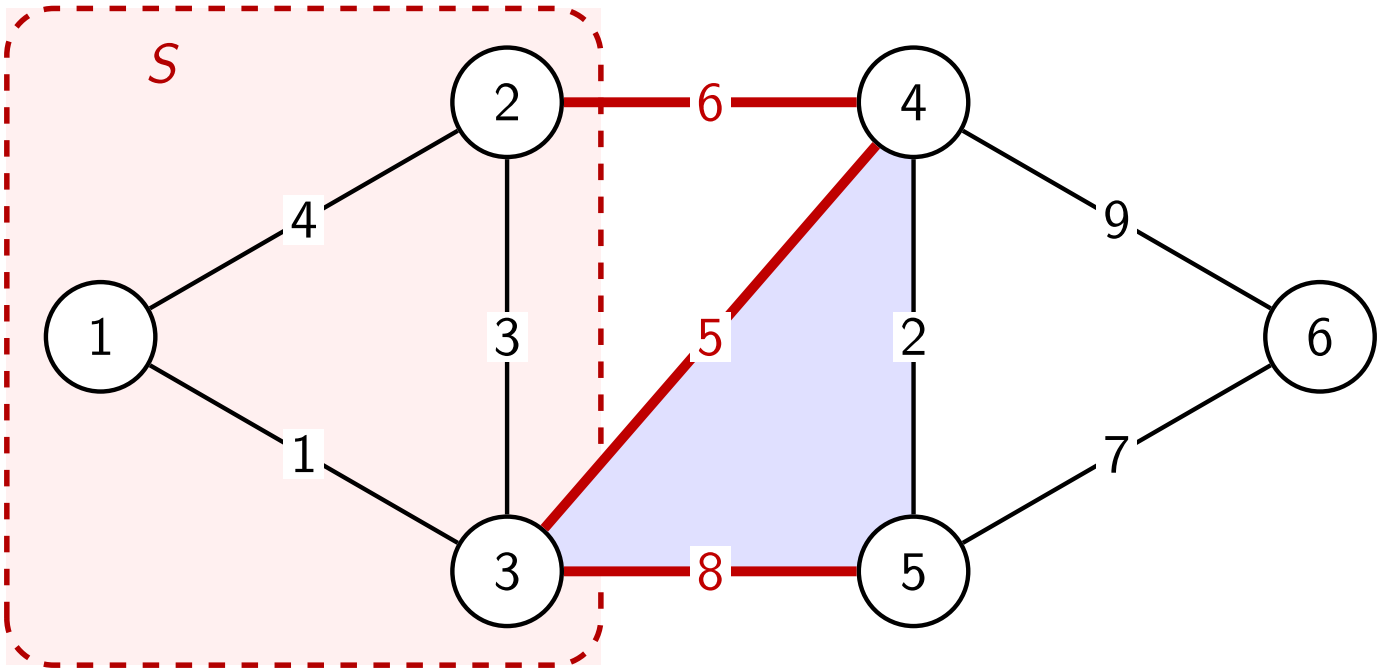
Part 2.

- The endpoints of e lie on different sides of the fundamental cut $\delta(S_f)$
- The path between them in T is part of $C(T, e)$, and it must cross $\delta(S_f)$ with an edge of T
- The only one is f, so $f \in C(T, e)$
- Part 1 concludes



Example: cuts and cycles of the reference graph (1/2)

The figure shows the cut $\delta(\{1,2,3\})$ (red edges) and the cycle $3,4,5$ (shaded in blue).



The cycle uses the edges $\{3,4\}$ and $\{3,5\}$ of the cut, an even number.

Example: cuts and cycles of the reference graph (2/2)

For the minimum spanning tree
$\tilde{F} = \big\{\{1,3\},\{2,3\},\{3,4\},\{4,5\},\{5,6\}\big\}$ of the reference graph:

- the fundamental cut of $\{3,4\}$ is

$$\delta(\{1,2,3\})$$

- the fundamental cycle of $\{3,5\}$ is

$$C(\tilde{F},\{3,5\}) = 3,4,5,3$$

Cuts, cycles and exchanges

A cycle crosses every cut an even number of times.

Adding an edge to a spanning tree and removing any edge of the cycle it closes gives another spanning tree.

Summary

Cuts:

- $\delta(S)$: the edges with exactly one endpoint in S; G is connected iff every cut is nonempty
- The fundamental cut of $e \in T$ contains e and no other edge of T

Cycles and exchanges:

- A cycle crosses a cut an even number of times
- Exchange lemma: $T \cup \{e\} \setminus \{f\}$ is a spanning tree when $f \in C(T, e)$

The cut property and the cycle property

Introduction: The cut property and the cycle property

- Two simple rules tell which edges are safe to take and which can be left out
- The **cut property**: the lightest edge of a cut; the **cycle property**: the heaviest edge of a cycle
- Together they give the optimality conditions: a certificate that a spanning tree is optimal, without enumerating all the spanning trees
- All the algorithms of the chapter rest on these results

Theorem 1: cut property

Let $G = (V, E)$ be a connected graph with weights w_e, and let $F \subseteq E$ be contained in some minimum spanning tree. Let $\delta(S)$ be a cut with $F \cap \delta(S) = \emptyset$, and let e be an edge of minimum weight in $\delta(S)$.

(a) *$F \cup \{e\}$ is contained in some minimum spanning tree.*

(b) *If $w_e < w_f$ for every $f \in \delta(S) \setminus \{e\}$, then e belongs to every minimum spanning tree.*

The proof is an exchange:
if an optimal tree does not contain e, we swap e with another edge of the cut, and the weight does not grow.

Proof (1/2)

- Let $T^\star$ be a minimum spanning tree with $F \subseteq T^\star$; for (b), let $T^\star$ be any minimum spanning tree and $F = \emptyset$
- If $e \in T^\star$ there is nothing to prove; let $e \notin T^\star$

1. The fundamental cycle $C(T^\star, e)$ contains the edge e of $\delta(S)$, so it contains another edge $f \in \delta(S)$, $f \neq e$ (Lemma 1). All its edges other than e are in $T^\star$, so $f \in T^\star$; and $f \notin F$, since $F \cap \delta(S) = \emptyset$.

Proof (2/2)

2. By Lemma 2, $T' = T^\star \cup \{e\} \setminus \{f\}$ is a spanning tree, with

$$w(T') = w(T^\star) + w_e - w_f$$

3. (a) e has minimum weight in $\delta(S)$, so $w_e \leq w_f$ and $w(T') \leq w(T^\star)$: T' is a minimum spanning tree containing $F \cup \{e\}$.

4. (b) Here $w_e < w_f$ and $w(T') < w(T^\star)$, against the optimality of $T^\star$: hence $e \in T^\star$.

□

The cycle property is the mirror image of the cut property:
it tells which edges can be left out.

Theorem 2: cycle property

Let $G = (V, E)$ be a connected graph with weights w_e, let C be a cycle of G and e an edge of maximum weight in C.

(a) *Some minimum spanning tree does not contain e.*

(b) *If $w_e > w_f$ for every $f \in C \setminus \{e\}$, then no minimum spanning tree contains e.*

Proof (1/2)

- Let $T^\star$ be a minimum spanning tree with $e \in T^\star$ (if there is none, both claims hold)
- Let $\delta(S_e)$ be the fundamental cut of e in $T^\star$

1. The cycle C contains the edge e of $\delta(S_e)$, so it contains another edge $f \in \delta(S_e)$ (Lemma 1); $f \notin T^\star$, because e is the only edge of $T^\star$ in its fundamental cut.

Proof (2/2)

2. By Lemma 2(2), $T' = T^\star \cup \{f\} \setminus \{e\}$ is a spanning tree, with

$$w(T') = w(T^\star) + w_f - w_e$$

3. (a) $w_f \leq w_e$, so T' is a minimum spanning tree without e.

   (b) $w_f < w_e$ would give $w(T') < w(T^\star)$, a contradiction: no minimum spanning tree contains e.

□

Example: the two properties on the reference graph (1/3)

Cut property.

- The edge $\{3, 4\}$, of weight 5, is the unique lightest edge of

$$\delta(\{1, 2, 3\}) = \{\{2, 4\}, \{3, 4\}, \{3, 5\}\},$$

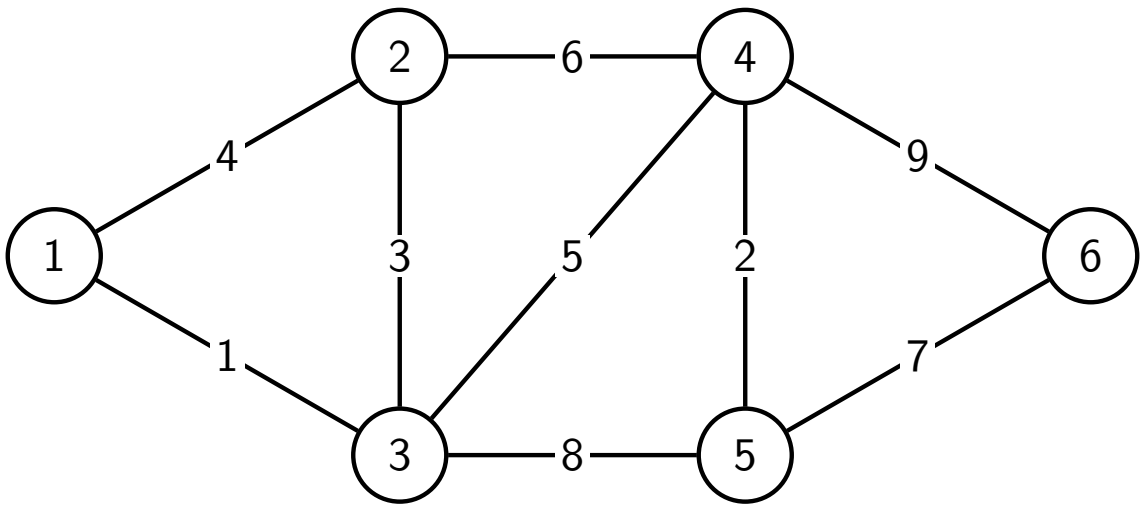
  with weights $6, 5, 8$: it belongs to every minimum spanning tree.

- In the same way, $\{5, 6\}$ is the unique lightest edge of $\delta(\{6\})$.

Example: the two properties on the reference graph (2/3)

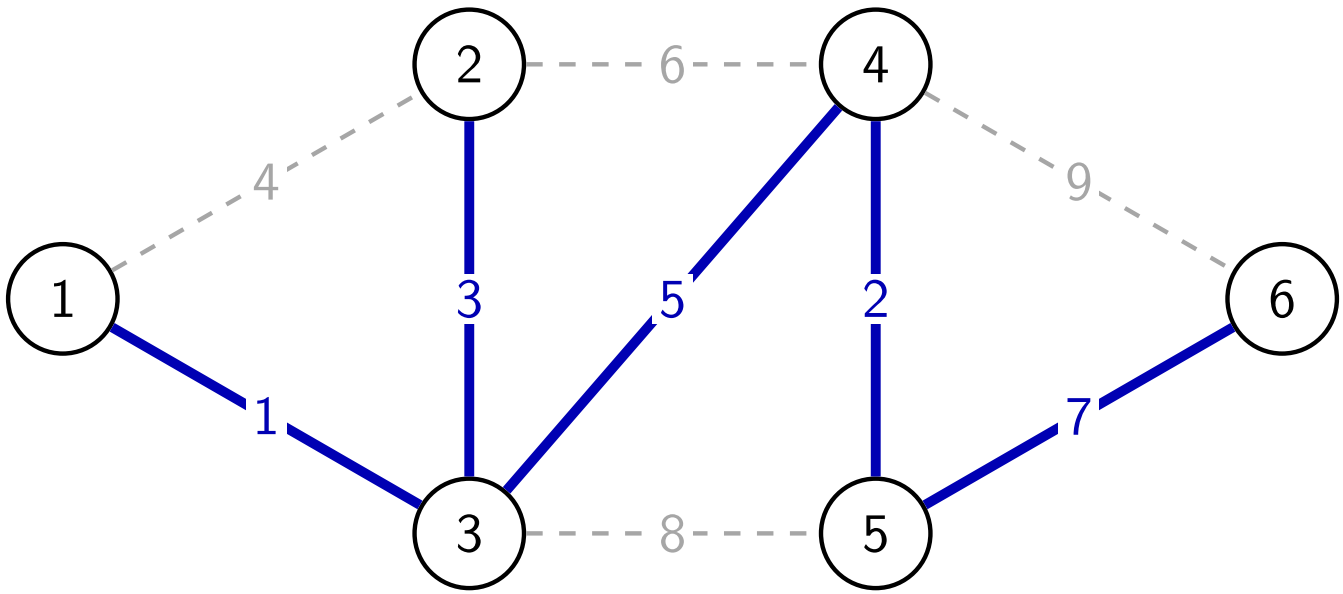
Cycle property.

- The edge $\{1, 2\}$ is the unique heaviest edge of the cycle $1, 2, 3$ (weights $4, 3, 1$)
- $\{3, 5\}$ is the unique heaviest edge of the cycle $3, 4, 5$ (weights $5, 2, 8$)
- No minimum spanning tree contains them
- The cycles $2, 3, 4$ and $4, 5, 6$ exclude $\{2, 4\}$ and $\{4, 6\}$



Example: the two properties on the reference graph (3/3)

- The four excluded edges leave exactly the 5 edges of $\tilde{F}$
- The two properties alone prove that $\tilde{F}$ is the unique minimum spanning tree



The two properties become a test of optimality:
for a given tree, we only check its fundamental cycles, or its fundamental cuts.

Theorem 3: optimality conditions

Let T be a spanning tree of a connected graph $G = (V, E)$ with weights w_e. The following statements are equivalent:

(i) *T is a minimum spanning tree;*

(ii) ***cycle condition**: for every $e \notin T$, $w_e \geq w_f$ for every edge f of $C(T, e)$;*

(iii) ***cut condition**: for every $e \in T$, $w_e \leq w_f$ for every edge f of the fundamental cut of e.*

Proof (1/3)

We prove (i)⇒(iii)⇒(ii)⇒(i).

- **(i)⇒(iii).** If f crosses the fundamental cut of $e \in T$ with $w_f < w_e$, then $T \cup \{f\} \setminus \{e\}$ is a spanning tree (Lemma 2(2)) lighter than T.

- **(iii)⇒(ii).** Let $e \notin T$ and $f \in C(T, e)$, $f \neq e$. The path of T between the endpoints of e uses f, and it is unique, so removing f separates them: e crosses the fundamental cut of f, and $w_f \leq w_e$ by (iii).

Proof (2/3)

(ii)⇒(i). Among the minimum spanning trees, let $T^\star$ have the largest number of edges in common with T, and assume $T^\star \neq T$.

1. Take $e \in T^\star \setminus T$, and let $\delta(S_e)$ be its fundamental cut in $T^\star$.

   The cycle $C(T, e)$ contains e, so it contains another edge $f \in \delta(S_e)$ (Lemma 1), with $f \in T$ and $f \notin T^\star$.

Proof (3/3)

2. By Lemma 2(2), $T' = T^\star \cup \{f\} \setminus \{e\}$ is a spanning tree, and $w_f \leq w_e$ by (ii) applied to $e \notin T$. Hence

$$w(T') \leq w(T^\star)$$

3. T' is a minimum spanning tree with one more edge in common with T: a contradiction. So $T = T^\star$.

□

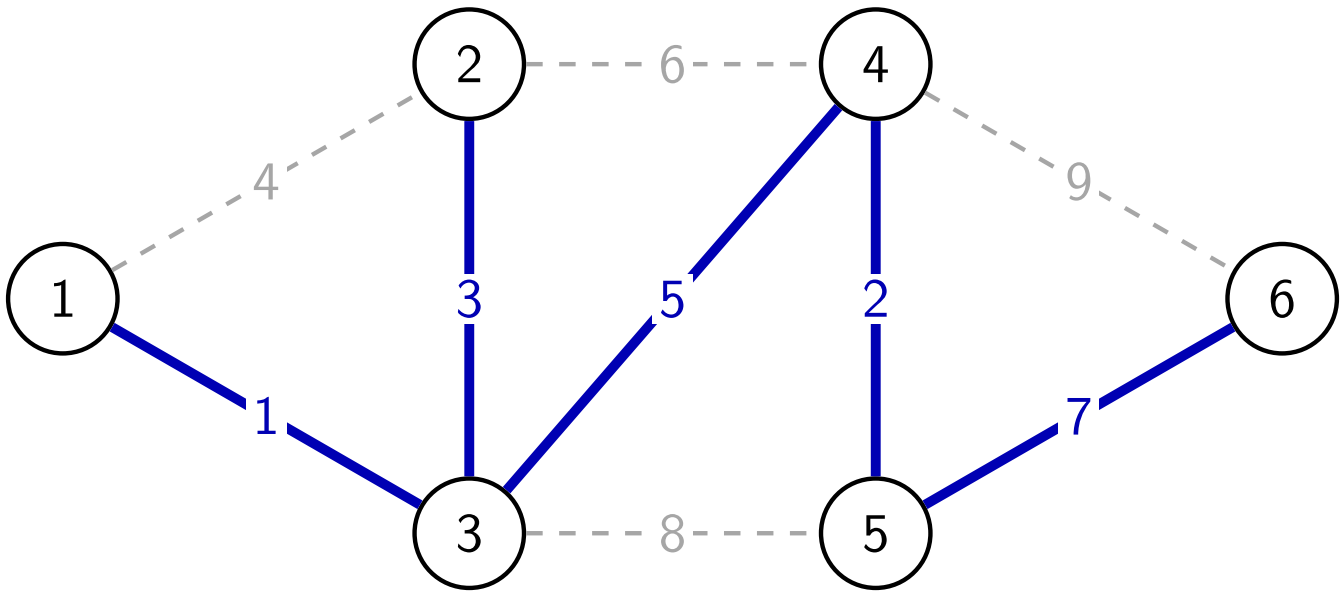
Example: certifying the optimal tree (1/2)

For the tree $\tilde{F}$ of the reference graph, the four edges outside the tree and their fundamental cycles are:

edge $e \notin \tilde{F}$	$C(\tilde{F}, e)$	weights of the tree edges of the cycle
$\{1,2\}$, $w = 4$	$1, 3, 2, 1$	$1,\ 3$
$\{2,4\}$, $w = 6$	$2, 3, 4, 2$	$3,\ 5$
$\{3,5\}$, $w = 8$	$3, 4, 5, 3$	$5,\ 2$
$\{4,6\}$, $w = 9$	$4, 5, 6, 4$	$2,\ 7$

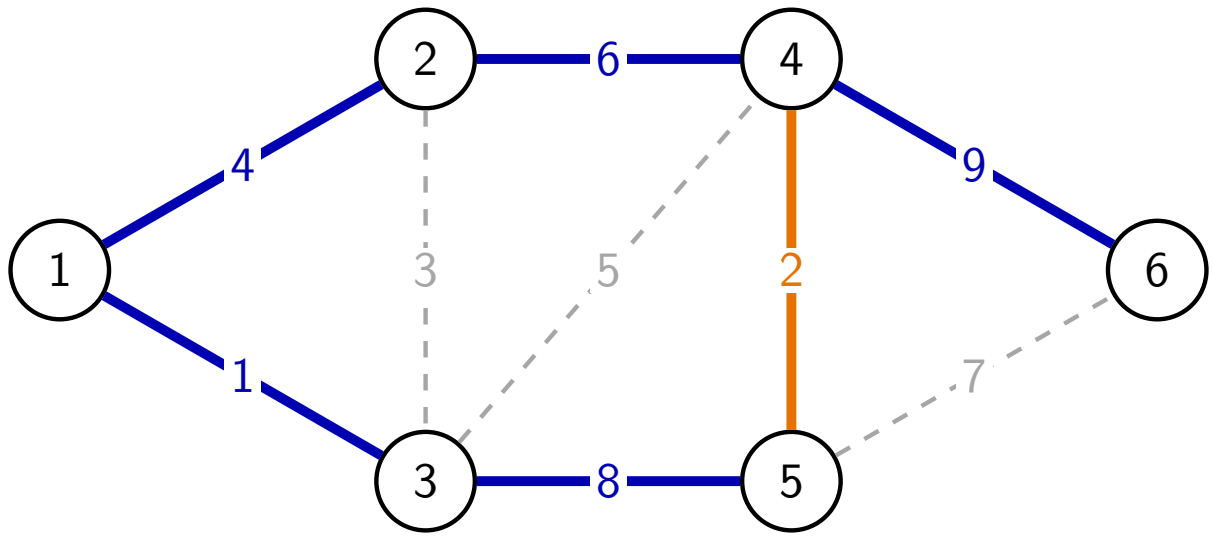
Example: certifying the optimal tree (2/2)

- Every edge outside the tree is at least as heavy as the tree edges of its cycle
- By Theorem 3 the tree is optimal, without enumerating the 55 spanning trees



Example: improving a tree that is not optimal (1/2)

- The BFS tree $T = \{\{1,2\},\{1,3\},\{2,4\},\{3,5\},\{4,6\}\}$, of weight 28, violates the cycle condition
- For $e = \{4,5\}$, of weight 2, the fundamental cycle $4,2,1,3,5,4$ contains $\{2,4\}$, $\{1,2\}$ and $\{3,5\}$, of weights $6,4,8$



blue: the BFS tree T; orange: the edge $e = \{4,5\}$

Example: improving a tree that is not optimal (2/2)

- Removing the heaviest of them, $\{3, 5\}$, and adding $\{4, 5\}$ gives a spanning tree of weight

$$28 + 2 - 8 = 22$$

- The proof of Theorem 3 turns into an algorithm: repeat the exchange while an edge violates the condition

- The algorithms of the next sections are faster

Cut, cycle and optimality

The lightest edge of a cut belongs to some minimum spanning tree, and the heaviest edge of a cycle can be left out.

A spanning tree is optimal if and only if every edge outside it is the heaviest of its fundamental cycle.

Summary

Cut and cycle properties:

- The lightest edge of a cut not crossed by F extends F within some minimum spanning tree
- The heaviest edge of a cycle can be left out; if strictly the heaviest, it is in no optimum

Optimality conditions:

- Optimal $\iff$ every edge outside T is the heaviest of its fundamental cycle
- $\iff$ every tree edge is the lightest of its fundamental cut

The generic greedy algorithm

Introduction: The generic greedy algorithm

- The cut property turns directly into an algorithm
- We describe a generic greedy algorithm and prove that it always returns a minimum spanning tree
- Kruskal's and Prim's algorithms are two special cases: proving the generic one correct proves both

The idea

- The cut property suggests building a minimum spanning tree one edge at a time
- The current set F is always contained in some minimum spanning tree
- The next edge is the lightest edge of a cut not crossed by F

Algorithm: Generic greedy algorithm for the MST

Input : a connected undirected graph $G = (V, E)$ with weights w_e

Output: a minimum spanning tree F

1. **Step 0 (initialization).** Set $F := \emptyset$.
2. **Step 1 (stopping test).** If $|F| = n - 1$, `STOP`.
3. **Step 2 (augmentation).** Choose a cut $\delta(S)$ with $F \cap \delta(S) = \emptyset$, and add to F an edge of minimum weight in $\delta(S)$. Go to Step 1.

Theorem 4: correctness of the generic greedy algorithm

At every iteration of the generic greedy algorithm a cut as in Step 2 exists and is nonempty, and F is contained in some minimum spanning tree. At the end, F is a minimum spanning tree.

Proof (1/2)

By induction on the iterations; $F = \emptyset$ is contained in every minimum spanning tree.

1. If $|F| < n - 1$, the graph (V, F) is a forest (a subset of a tree) with $n - |F| \geq 2$ components.

   Let S be one of them: no edge of F crosses $\delta(S)$, and $\delta(S) \neq \emptyset$ because G is connected.

Proof (2/2)

2. By the cut property (Theorem 1(a)), the new set $F \cup \{e\}$ is still contained in some minimum spanning tree.

3. When $|F| = n - 1$, F is contained in a minimum spanning tree $T^\star$ with $n - 1$ edges, so $F = T^\star$.

□

Remarks

- The added edge $e = \{u, v\}$ has $u \in S$ and $v \notin S$, and no edge of F crosses $\delta(S)$
- So u and v are in different components of (V, F), and e does not close a cycle
- The algorithms of Kruskal and of Prim are two ways of choosing the cut in Step 2

The greedy principle

Adding, $n-1$ times, the lightest edge of a cut not crossed by the current edges builds a minimum spanning tree.

Summary

Generic greedy algorithm:

- Start from $F = \emptyset$ and add $n - 1$ times the lightest edge of a cut not crossed by F
- A suitable cut always exists: a component of the forest (V, F)

Correctness:

- By the cut property, F always stays inside some minimum spanning tree
- Kruskal and Prim are two ways of choosing the cut

Kruskal's algorithm

Introduction: Kruskal's algorithm

- Kruskal's algorithm scans the edges from the lightest to the heaviest, and keeps an edge whenever it does not close a cycle
- We prove that it is a special case of the generic greedy algorithm
- With a union–find structure it runs in $O(m \log n)$ time
- We run it step by step on the weighted reference graph

Algorithm: Kruskal's algorithm

Input : a connected undirected graph $G = (V, E)$ with weights w_e

Output: a minimum spanning tree F

1 **Step 0 (initialization).** Sort the edges by non-decreasing weight, $w_{e_1} \leq w_{e_2} \leq \cdots \leq w_{e_m}$, and set $F := \emptyset$.

2 **Step 1 (scan).** For $k = 1, 2, \ldots, m$: if the endpoints of e_k are in different components of (V, F), set $F := F \cup \{e_k\}$; otherwise reject e_k.

3 If $|F| = n - 1$, STOP.

Accepted and rejected edges

- An edge is **accepted** if it joins two different components of (V, F), and **rejected** if it would close a cycle
- The components of (V, F) start as the n single vertices and merge at every acceptance
- In this module, edges of equal weight are scanned in lexicographic order of $\{u, v\}$, with $u < v$

Theorem 5: correctness of Kruskal's algorithm

Kruskal's algorithm stops with $|F| = n - 1$, and F is a minimum spanning tree.

- The approach: every accepted edge is a choice of Step 2 of the generic greedy algorithm

Proof (1/3)

- We show that every accepted edge $e_k = \{u, v\}$ is a choice of Step 2 of the generic greedy algorithm
- Let S be the component of (V, F) that contains u, just before e_k is accepted
- Then $v \notin S$, and no edge of F crosses $\delta(S)$

Proof (2/3)

1. Let $e_h \in \delta(S)$ with $h < k$. When e_h was scanned it was not accepted, since $F \cap \delta(S) = \emptyset$; so its endpoints were in the same component then, and they still are, because components only merge. This contradicts $e_h \in \delta(S)$.

2. Hence every edge of $\delta(S)$ is some e_h with $h \geq k$, and $w_{e_h} \geq w_{e_k}$: e_k has minimum weight in $\delta(S)$.

Proof (3/3)

3. If the scan ended with $|F| < n - 1$, the forest (V, F) would have two components joined by an edge of G, and that edge would have been accepted when scanned, by the argument of point 1.

By Theorem 4, F is a minimum spanning tree. □

The union–find structure

- The components of (V, F) are kept in a **union–find** structure:
  - every vertex v stores the label $comp(v)$ of its component;
  - every component stores the list of its vertices and its size.
- Testing whether u and v are in different components is the comparison

$$comp(u) \neq comp(v),$$

  in $O(1)$ time
- Merging two components relabels the vertices of the *smaller* one (**union by size**)

Theorem 6: running time of Kruskal's algorithm

With the union–find structure by labels and union by size, Kruskal's algorithm runs in $O(m \log n)$ time.

Proof (1/2)

1. Sorting the edges takes $O(m \log m)$ time; since $m \leq n^2$, $\log m \leq 2 \log n$ and this is $O(m \log n)$.

2. The scan performs m tests, each in $O(1)$ time.

3. When a vertex is relabelled, its component merges with one at least as large, so the size of its component at least doubles.

Proof (2/2)

- The size never exceeds n, so each vertex is relabelled at most $\log_2 n$ times: all the merges take $O(n \log n)$ time
- G is connected, so $n - 1 \leq m$, and the total is

$$O(m \log n + n \log n) = O(m \log n)$$

□

Remarks on the running time

- The running time is dominated by the sorting
- With the union–find structure by trees, with union by rank and path compression, the scan takes almost linear time; the bound stays $O(m \log n)$ because of the sorting
- The algorithm uses only comparisons between weights: any increasing transformation of the weights gives the same tree

Example: Kruskal's algorithm (1/10)

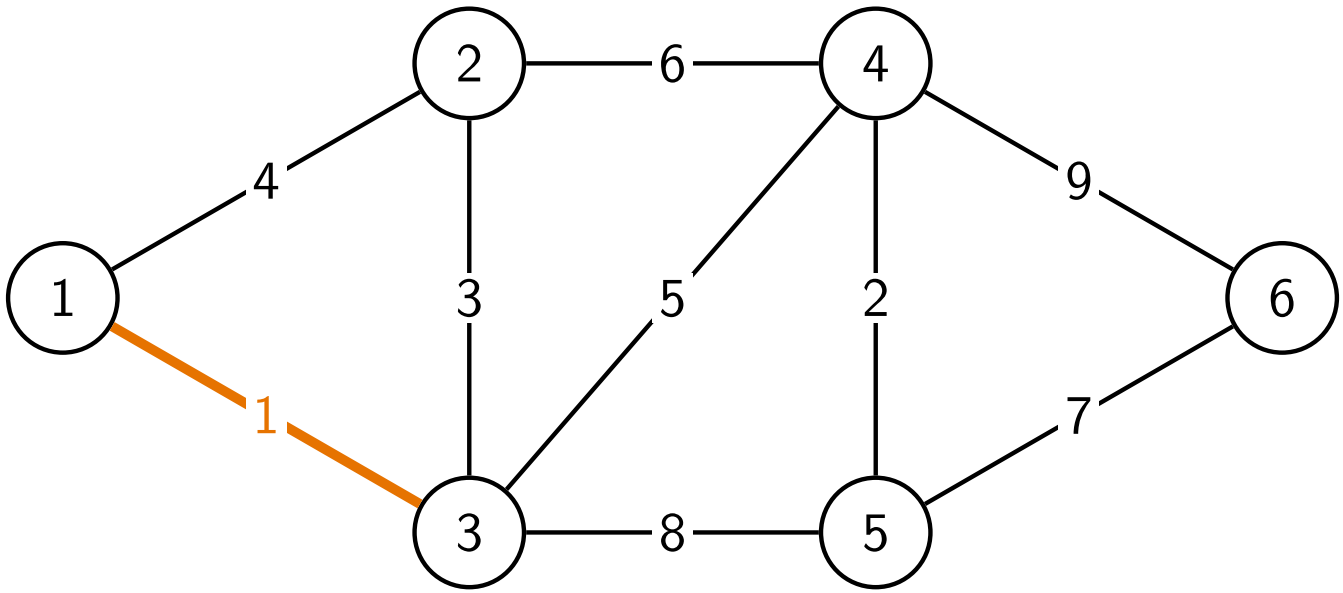
- We run Kruskal's algorithm on the weighted reference graph
- The edges in order of weight are

$$\{1,3\},\{4,5\},\{2,3\},\{1,2\},\{3,4\},\{2,4\},\{5,6\},\{3,5\},\{4,6\}$$

- In the next slides, one slide per scanned edge:
  - **blue**: the accepted edges, in F;
  - **orange**: the edge scanned now;
  - gray dashed: the rejected edges.

Example: Kruskal's algorithm (2/10)

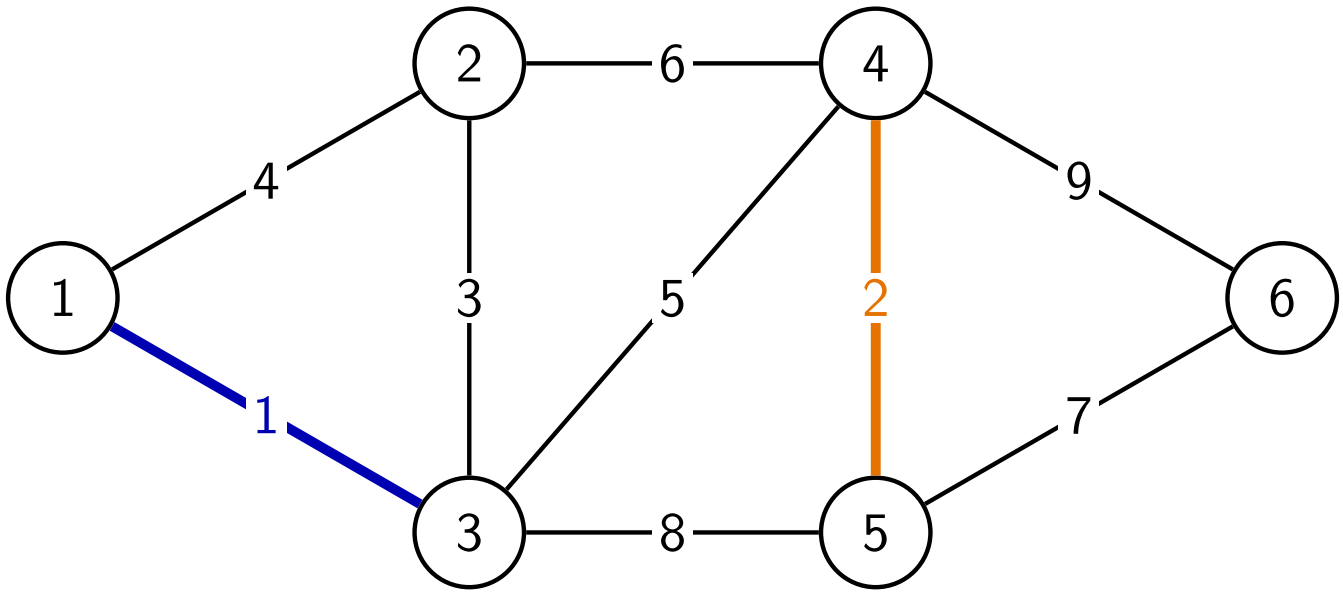
$k = 1$: edge $\{1, 3\}$, $w = 1$. **Accept**.



Components of (V, F): $\{1, 3\}, \{2\}, \{4\}, \{5\}, \{6\}$

Example: Kruskal's algorithm (3/10)

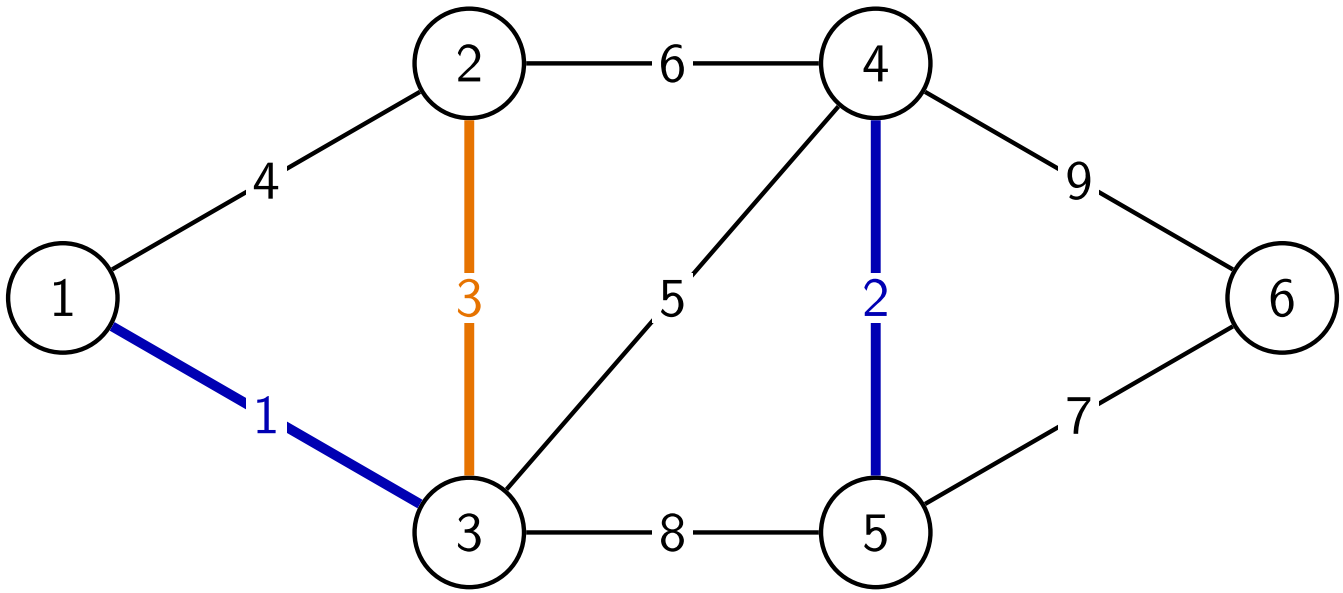
$k = 2$: edge $\{4, 5\}$, $w = 2$. **Accept**.



Components of (V, F): $\{1, 3\}, \{2\}, \{4, 5\}, \{6\}$

Example: Kruskal's algorithm (4/10)

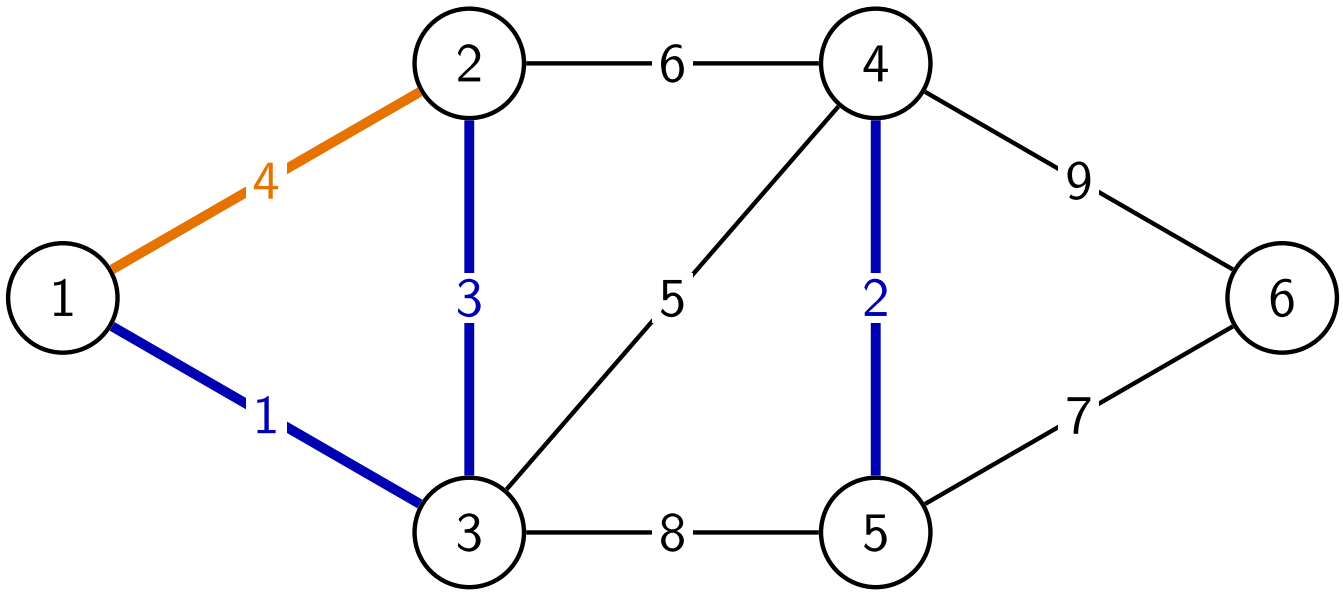
$k = 3$: edge $\{2, 3\}$, $w = 3$. **Accept**.



Components of (V, F): $\{1, 2, 3\}, \{4, 5\}, \{6\}$

Example: Kruskal's algorithm (5/10)

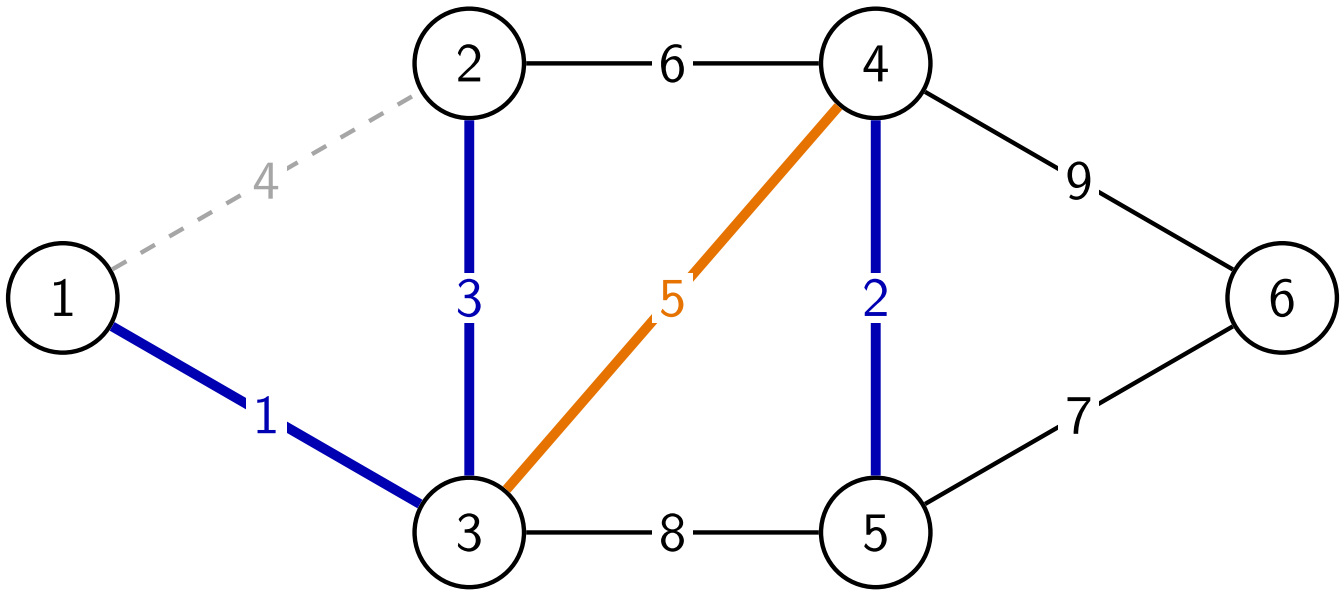
$k = 4$: edge $\{1, 2\}$, $w = 4$. **Reject**: 1 and 2 are both in $\{1, 2, 3\}$.



Components of (V, F): $\{1, 2, 3\}, \{4, 5\}, \{6\}$

Example: Kruskal's algorithm (6/10)

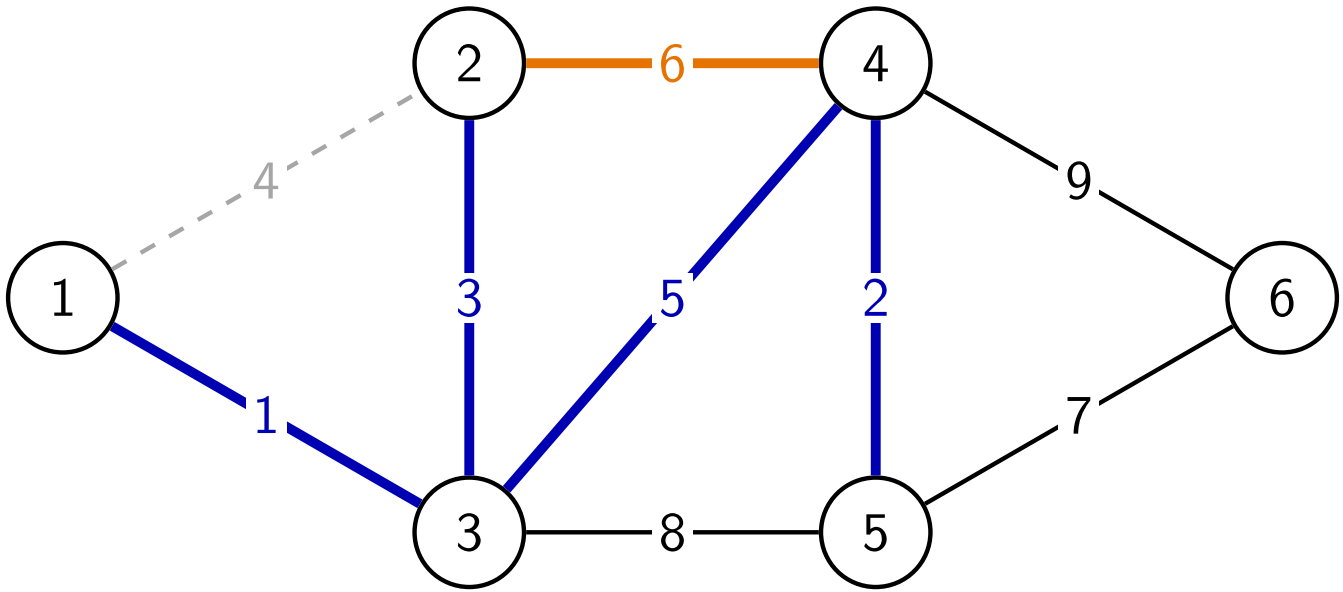
$k = 5$: edge $\{3, 4\}$, $w = 5$. **Accept**.



Components of (V, F): $\{1, 2, 3, 4, 5\}, \{6\}$

Example: Kruskal's algorithm (7/10)

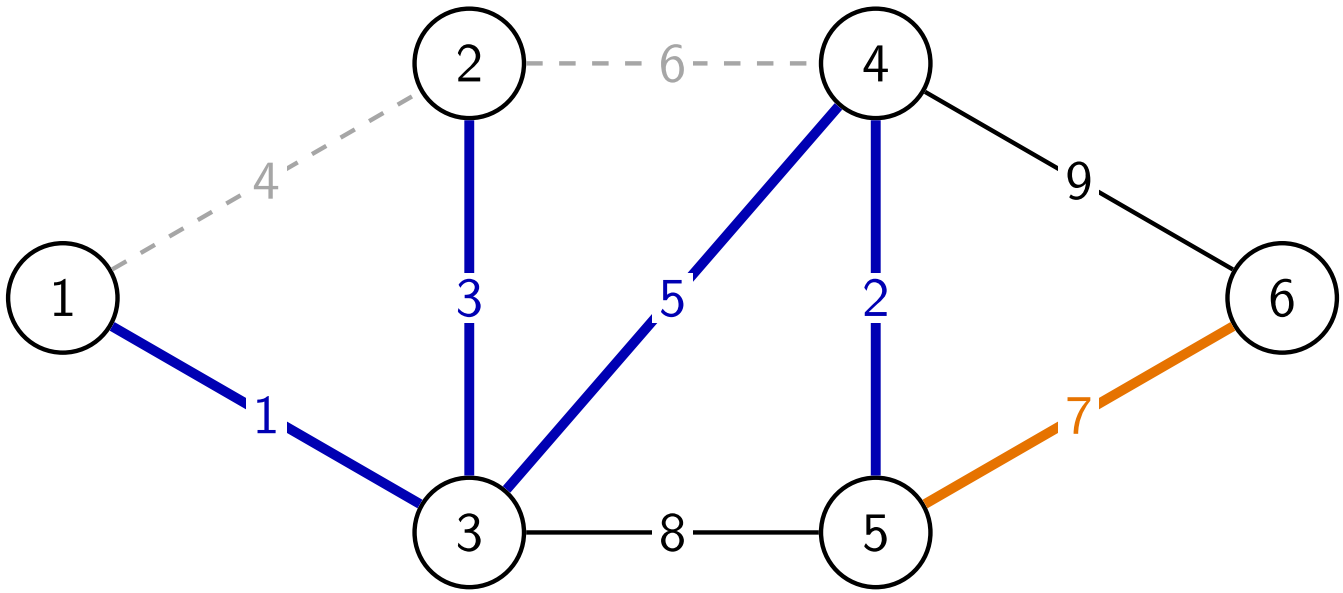
$k = 6$: edge $\{2, 4\}$, $w = 6$. **Reject**: 2 and 4 are both in $\{1, 2, 3, 4, 5\}$.



Components of (V, F): $\{1, 2, 3, 4, 5\}, \{6\}$

Example: Kruskal's algorithm (8/10)

$k = 7$: edge $\{5, 6\}$, $w = 7$. **Accept**: $|F| = 5$, STOP.



Components of (V, F): $\{1, 2, 3, 4, 5, 6\}$

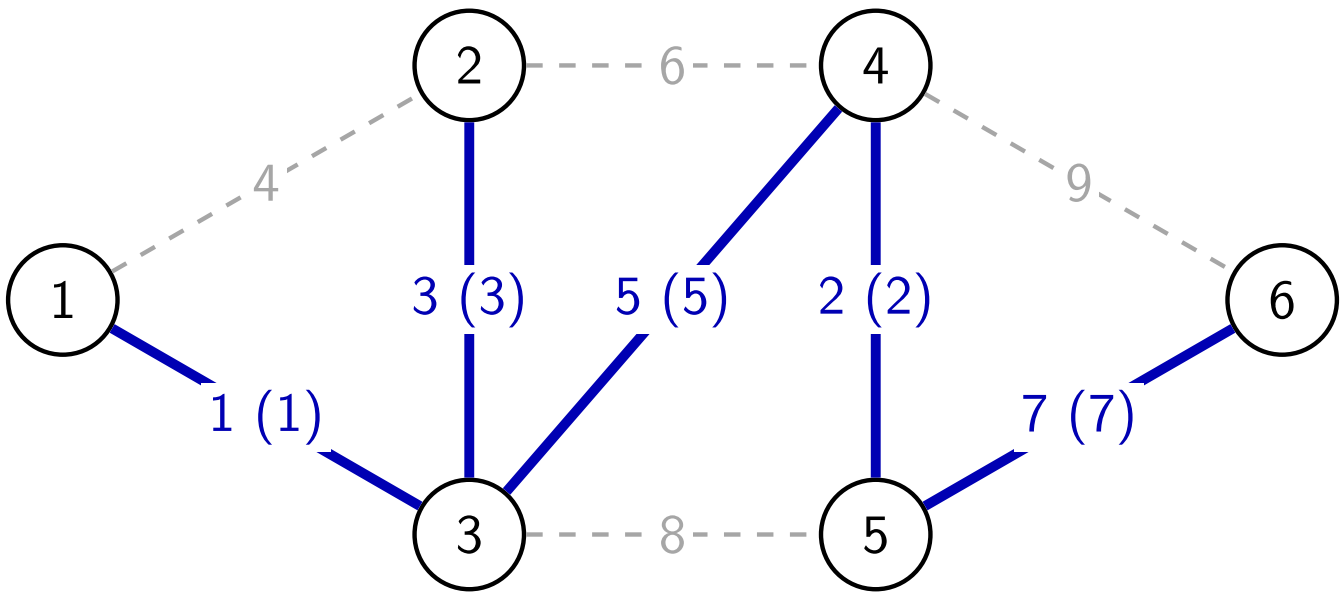
Example: Kruskal's algorithm (9/10)

Each row is one scanned edge, with the components of (V, F) after the step:

k	edge e_k	w_{e_k}	decision and components of (V, F)
1	$\{1,3\}$	1	accept: $\{1,3\},\{2\},\{4\},\{5\},\{6\}$
2	$\{4,5\}$	2	accept: $\{1,3\},\{2\},\{4,5\},\{6\}$
3	$\{2,3\}$	3	accept: $\{1,2,3\},\{4,5\},\{6\}$
4	$\{1,2\}$	4	reject (1 and 2 are both in $\{1,2,3\}$)
5	$\{3,4\}$	5	accept: $\{1,2,3,4,5\},\{6\}$
6	$\{2,4\}$	6	reject (2 and 4 are both in $\{1,2,3,4,5\}$)
7	$\{5,6\}$	7	accept: $\{1,2,3,4,5,6\}$; $\lvert F\rvert = 5$, STOP

Example: Kruskal's algorithm (10/10)

- The algorithm stops after 7 of the 9 edges, with $F = \{\{1,3\},\{4,5\},\{2,3\},\{3,4\},\{5,6\}\}$ of weight 18: the tree $\tilde{F}$
- On each tree edge: its weight and, in brackets, the step k of its acceptance



Kruskal and the cycle property

- The rejected edges $\{1, 2\}$ and $\{2, 4\}$ are the heaviest edges of the cycles $1, 2, 3$ and $2, 3, 4$, as the cycle property predicts
- The edges $\{3, 5\}$ and $\{4, 6\}$ are never scanned

Kruskal's algorithm in one sentence

Kruskal's algorithm scans the edges by non-decreasing weight and accepts those that join two components; with union–find it runs in $O(m \log n)$ time, dominated by the sorting.

Summary

Kruskal's algorithm:

- Sort the edges; accept an edge if it joins two components of (V, F), reject it otherwise
- Every acceptance is a choice of the generic greedy algorithm: the algorithm is correct

Running time:

- Union–find by labels with union by size: $O(m \log n)$, dominated by the sorting

On the reference graph:

- Accepts $\{1,3\}, \{4,5\}, \{2,3\}, \{3,4\}, \{5,6\}$; rejects $\{1,2\}, \{2,4\}$

Prim's algorithm

Introduction: Prim's algorithm

- Prim's algorithm grows a single tree from a start vertex, adding at every step the lightest edge that leaves it
- Labels on the vertices make the choice of that edge fast: $O(n^2)$ time in total
- We prove it correct as a special case of the generic greedy algorithm, and we run it on the weighted reference graph

The idea

- Prim's algorithm grows a single tree from a start vertex s
- The set S of the vertices of the tree defines the cut $\delta(S)$ of Step 2
- The lightest edge of the cut is found through a **label** $d(v)$ for every vertex $v \notin S$

Algorithm: Prim's algorithm

Input : a connected undirected graph $G = (V, E)$ with weights w_e, given by its adjacency lists; a start vertex $s \in V$

Output : a minimum spanning tree F

1. **Step 0 (initialization).** Set $S := \{s\}$ and $F := \emptyset$. For every $v \neq s$: if $\{s, v\} \in E$, set $d(v) := w_{sv}$ and $pred(v) := s$; otherwise set $d(v) := +\infty$.
2. **Step 1 (selection).** If $S = V$, `STOP`. Otherwise select $v \notin S$ with minimum $d(v)$, and set $S := S \cup \{v\}$, $F := F \cup \{\{pred(v), v\}\}$.
3. **Step 2 (update).** For every $u \notin S$ adjacent to v with $w_{vu} < d(u)$, set $d(u) := w_{vu}$ and $pred(u) := v$. Go to Step 1.

Conventions and the meaning of the labels

- In this module the start vertex is $s = 1$, and ties in Step 1 are broken by the smallest index
- The next lemma says what the labels are: the weight of the lightest edge from v to the current tree

Lemma 3: meaning of the labels

At every execution of Step 1 of Prim's algorithm, for every $v \notin S$,

$$d(v) = \min \left\{ w_{uv} : u \in S,\ \{u, v\} \in E \right\}$$

($+\infty$ *if* v *has no neighbour in* S*), and when* $d(v) < +\infty$ *the minimum is attained by* $u = pred(v)$.

Proof

By induction.

- At the first execution $S = \{s\}$, and Step 0 sets exactly $d(v) = w_{sv}$, or $+\infty$
- Assume the claim before the insertion of a vertex v into S
- For $u \notin S \cup \{v\}$, the new set $S \cup \{v\}$ adds the candidate w_{vu} when $\{v, u\} \in E$, and Step 2 replaces $d(u)$ and $pred(u)$ exactly when this candidate is smaller
- The other labels and predecessors are unchanged

□

Theorem 7: correctness of Prim's algorithm

Prim's algorithm stops with $|F| = n - 1$, and F is a minimum spanning tree.

Proof

All the edges of F have both endpoints in S, so $F \cap \delta(S) = \emptyset$.

1. By Lemma 3, $\min_{v \notin S} d(v)$ is the minimum weight of an edge of $\delta(S)$, attained by $\{pred(v), v\}$ for the selected v; it is finite because G is connected.

2. Hence every iteration is a choice of Step 2 of the generic greedy algorithm, with the cut $\delta(S)$.

3. Each iteration adds one vertex to S and one edge to F, so the algorithm stops after $n - 1$ iterations with $|F| = n - 1$; Theorem 4 concludes.

□

Theorem 8: running time of Prim's algorithm

With the labels stored in an array, Prim's algorithm runs in $O(n^2)$ time.

Proof

- There are $n-1$ selections, and each scans the labels of at most n vertices: $O(n^2)$ in total
- The update after the insertion of v scans the adjacency list of v in $O(\deg(v))$ time, and

$$\sum_{v \in V} \deg(v) = 2m$$

- The total is $O(n^2 + m) = O(n^2)$, since $m \leq n^2$

□

Dense and sparse graphs

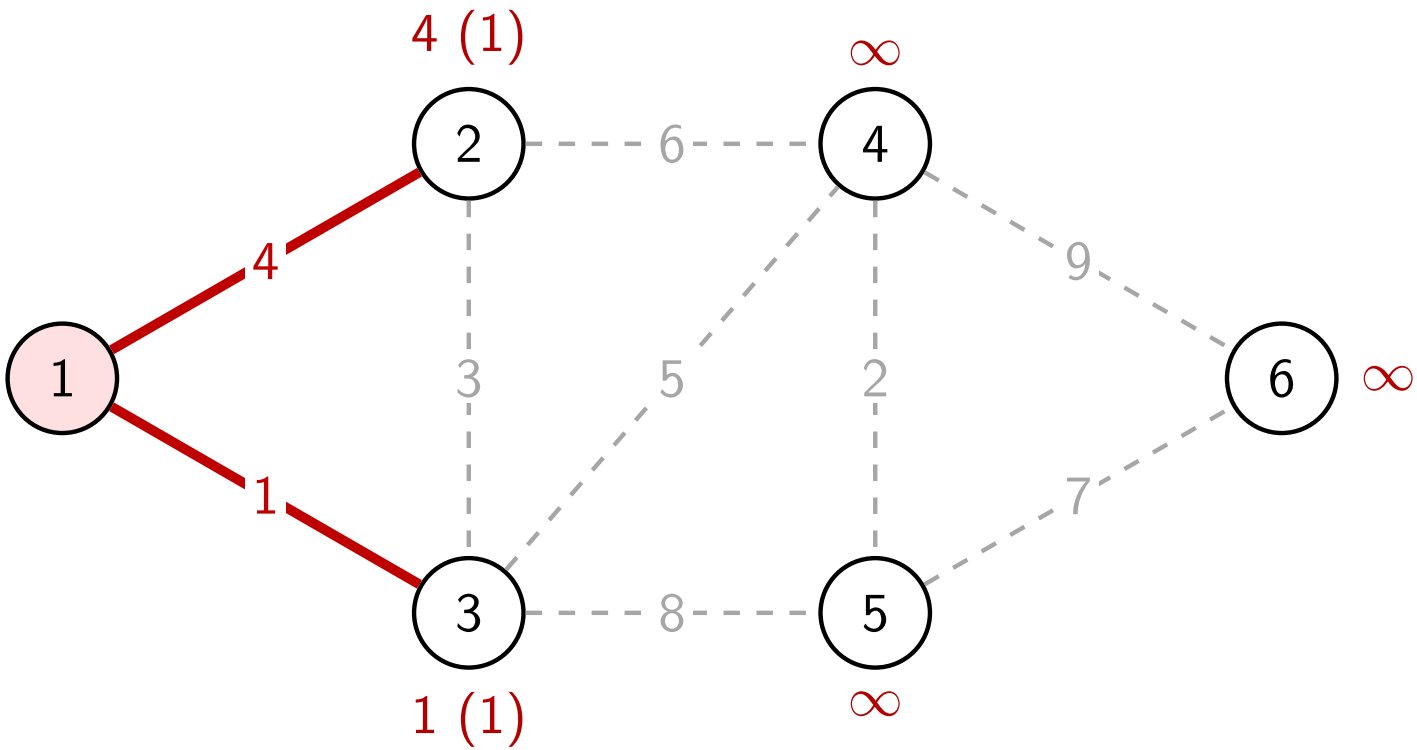
- On dense graphs, with m of the order of n^2, the bound $O(n^2)$ is linear in the size of the graph
- With the labels in a binary heap, each selection and each update take $O(\log n)$ time, for $O(m \log n)$ in total: better on sparse graphs

Example: Prim's algorithm from vertex 1 (1/10)

- We run Prim's algorithm on the weighted reference graph from $s = 1$
- In the next slides, one slide per iteration, after the update:
  - the vertices of S are shaded;
  - **blue**: the edges of F;
  - **red**: the edges of the cut $\delta(S)$;
  - next to every vertex $v \notin S$: its label $d(v)$, with $pred(v)$ in brackets.

Example: Prim's algorithm from vertex 1 (2/10)

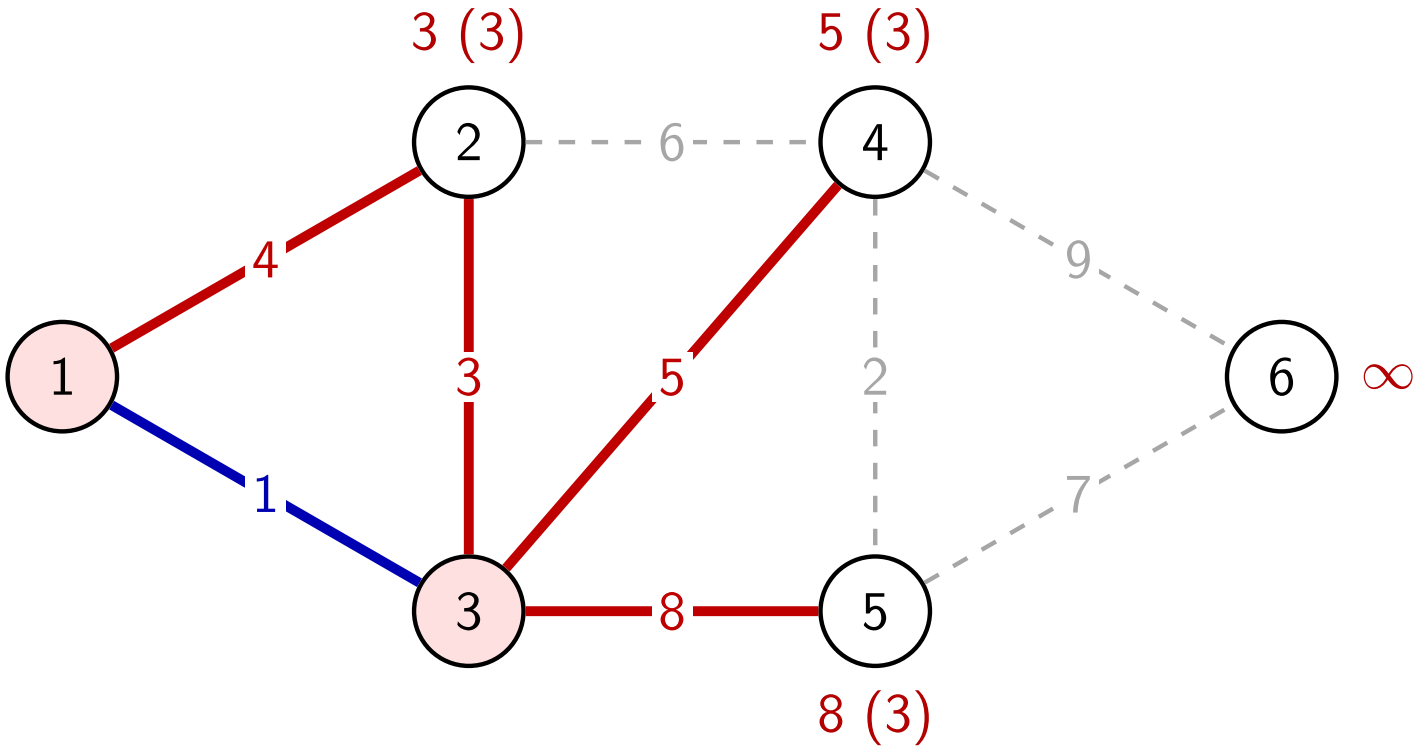
Iteration 0: $S = \{1\}$, $F = \emptyset$.



The next vertex is 3, with $d(3) = 1$.

Example: Prim's algorithm from vertex 1 (3/10)

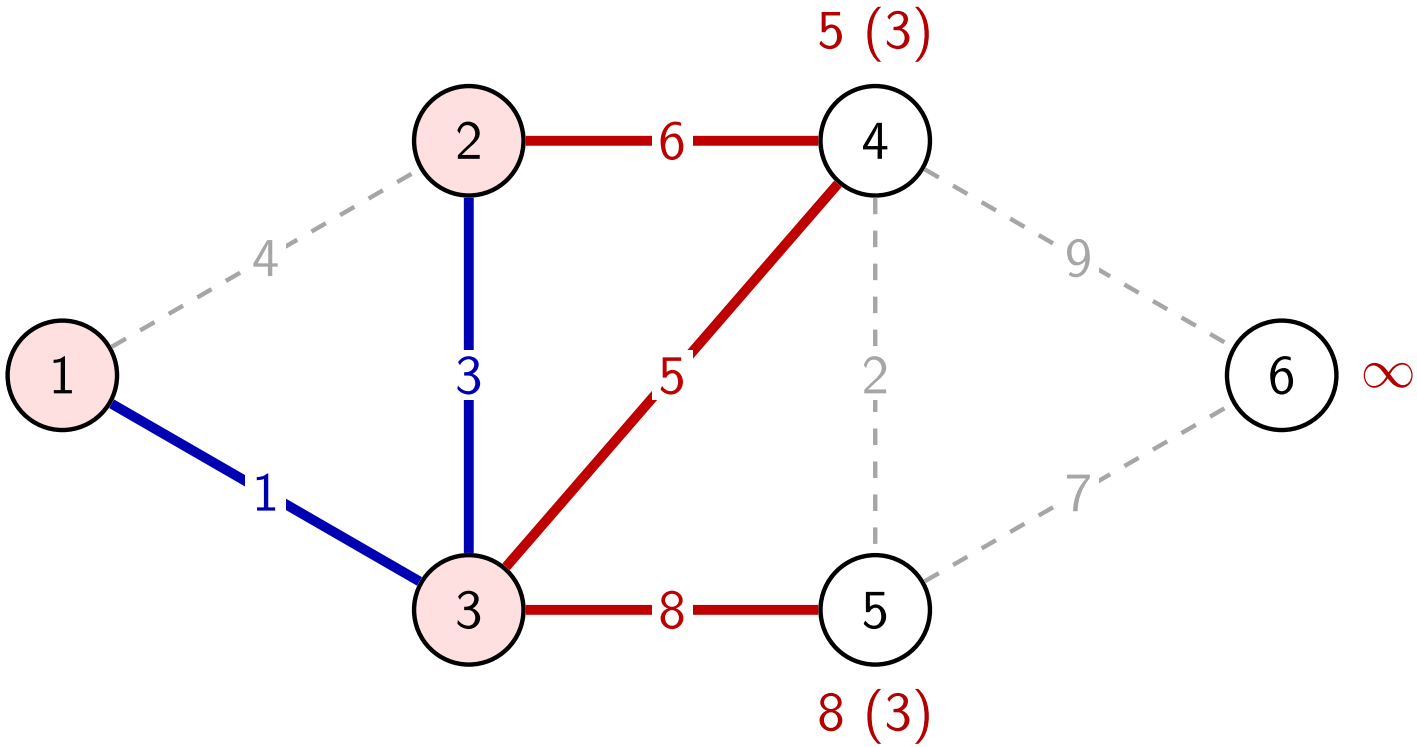
Iteration 1: select 3, add $\{1, 3\}$.



The label of 2 drops from 4 to 3, because $w_{32} = 3 < 4$.

Example: Prim's algorithm from vertex 1 (4/10)

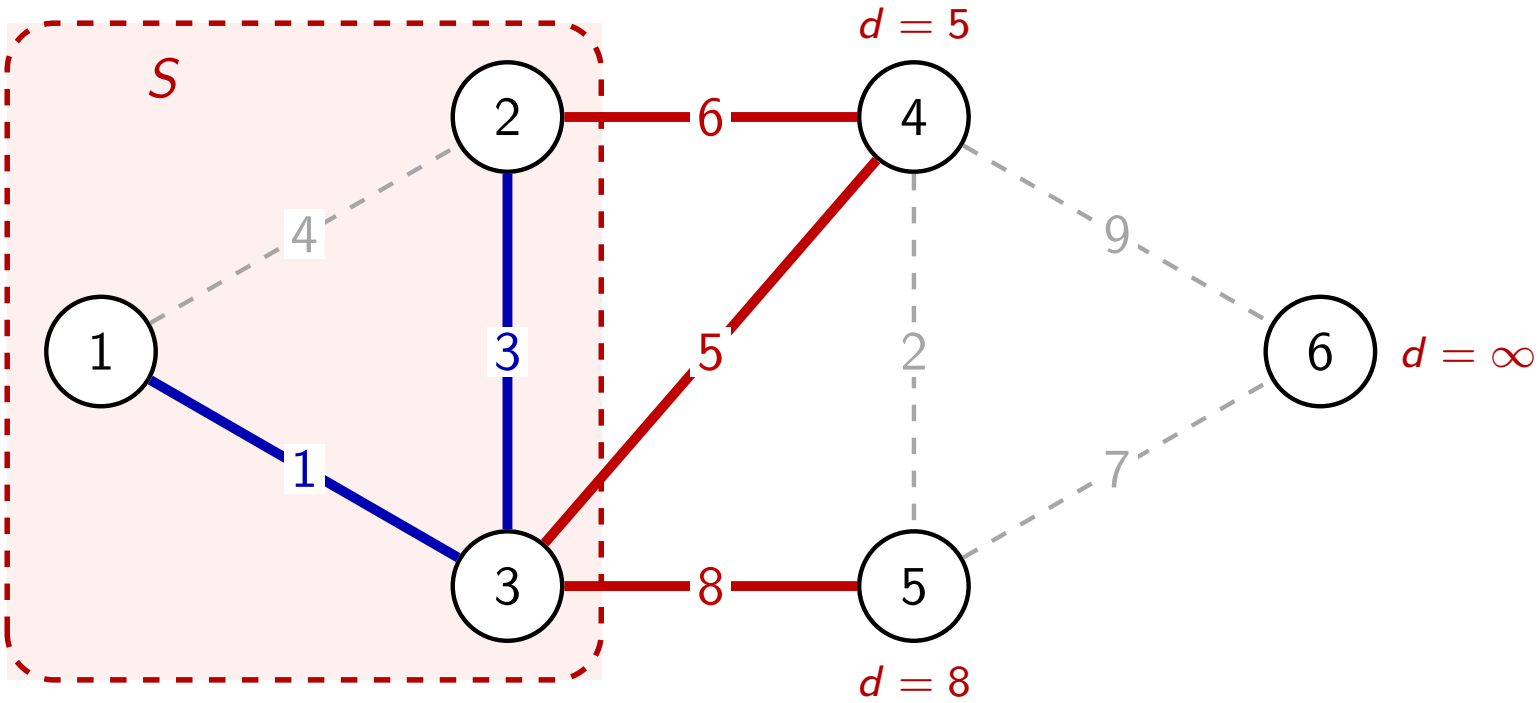
Iteration 2: select 2, add $\{2, 3\}$.



No label changes: $w_{24} = 6 > d(4) = 5$.

Example: Prim's algorithm from vertex 1 (5/10)

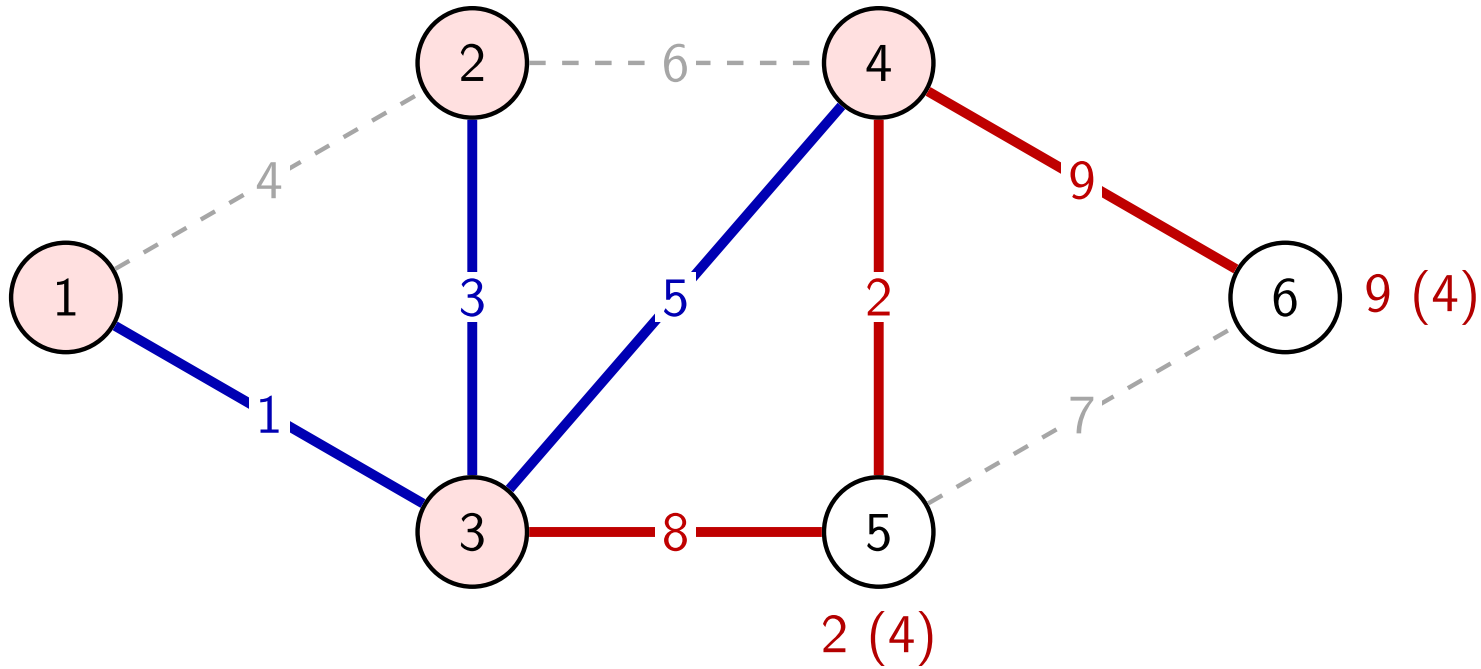
The state after iteration 2, with the set S and the cut $\delta(S)$:



The cut $\delta(\{1,2,3\})$ is crossed by $\{2,4\}$, $\{3,4\}$, $\{3,5\}$, and the next edge is $\{3,4\}$, with $d(4) = 5$.

Example: Prim's algorithm from vertex 1 (6/10)

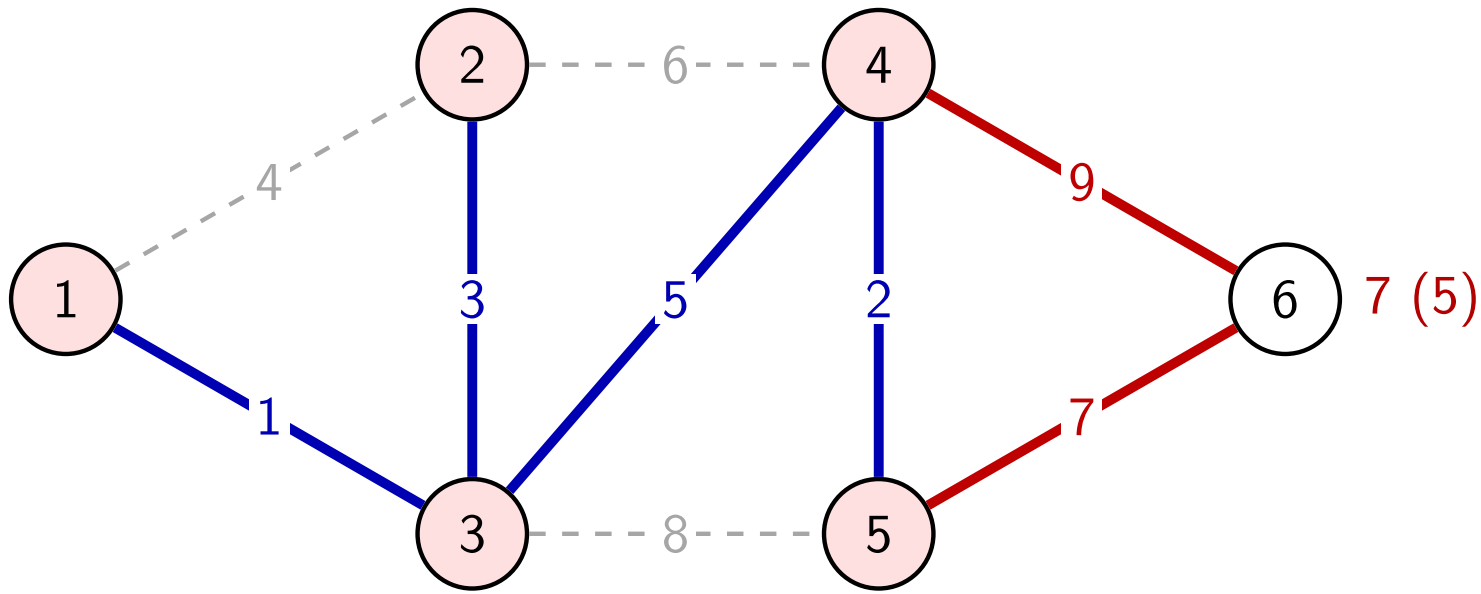
Iteration 3: select 4, add $\{3, 4\}$.



The label of 5 drops from 8 to 2.

Example: Prim's algorithm from vertex 1 (7/10)

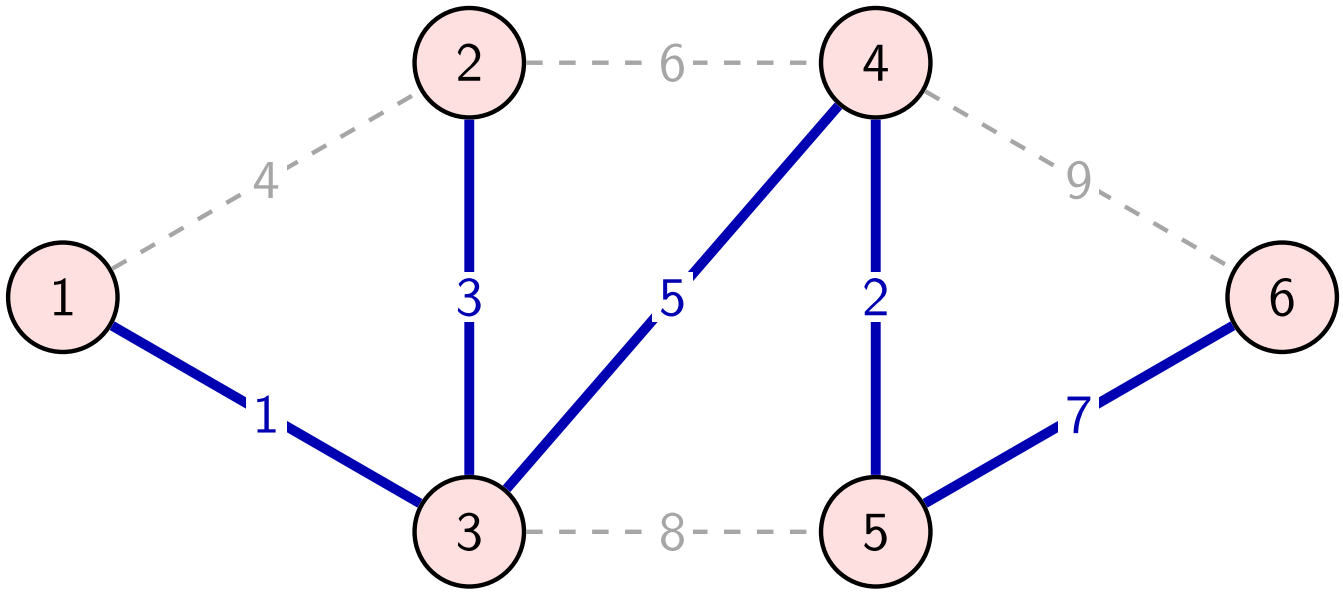
Iteration 4: select 5, add $\{4, 5\}$.



The label of 6 drops from 9 to 7.

Example: Prim's algorithm from vertex 1 (8/10)

Iteration 5: select 6, add $\{5, 6\}$. Now $S = V$: `STOP`.



The final tree weighs 18.

Example: Prim's algorithm from vertex 1 (9/10)

Each row: the selected vertex, the added edge and the labels after the update, with $pred(v)$ in brackets; a dash marks the vertices already in S.

iteration	selected	added edge	$d(2)$	$d(3)$	$d(4)$	$d(5)$	$d(6)$
0	1	–	4 (1)	1 (1)	∞	∞	∞
1	3	$\{1,3\}$	3 (3)	–	5 (3)	8 (3)	∞
2	2	$\{2,3\}$	–	–	5 (3)	8 (3)	∞
3	4	$\{3,4\}$	–	–	–	2 (4)	9 (4)
4	5	$\{4,5\}$	–	–	–	–	7 (5)
5	6	$\{5,6\}$	–	–	–	–	–

Example: Prim's algorithm from vertex 1 (10/10)

- The final tree is $\{1,3\},\{2,3\},\{3,4\},\{4,5\},\{5,6\}$, of weight 18
- It is the same tree as Kruskal's algorithm, with the edges added in a different order

Kruskal and Prim

- Kruskal's algorithm adds $\{4, 5\}$ second, while Prim's algorithm adds it fourth
- Kruskal grows a forest, Prim a single tree containing s

Prim's algorithm in one sentence

Prim's algorithm grows one tree from s and adds the lightest edge leaving it; the labels $d(v)$ give that edge in $O(n)$ time, for $O(n^2)$ in total.

Summary

Prim's algorithm:

- Grows one tree from s; the cut is $\delta(S)$, S the vertices of the tree
- $d(v)$ is the weight of the lightest edge from v to S, attained by $\{pred(v), v\}$

Running time:

- $O(n^2)$ with an array, good on dense graphs; $O(m \log n)$ with a binary heap

On the reference graph:

- From $s = 1$ it selects $3, 2, 4, 5, 6$: the same tree $\tilde{F}$ as Kruskal

Uniqueness of the minimum spanning tree

Introduction: Uniqueness of the minimum spanning tree

- The algorithms return one minimum spanning tree: are there others?
- With distinct weights the answer is no
- With ties we characterize the edges that belong to every, or to no, minimum spanning tree, and we decide whether the optimum is unique

Theorem 9: distinct weights

If the weights w_e, $e \in E$, are pairwise distinct, the connected graph $G = (V, E)$ has exactly one minimum spanning tree.

Proof (1/2)

- Assume that $T_1 \neq T_2$ are minimum spanning trees
- They have the same number of edges, so the symmetric difference

$$(T_1 \setminus T_2) \cup (T_2 \setminus T_1)$$

  is nonempty
- Let e be its lightest edge, and say $e \in T_1 \setminus T_2$

Proof (2/2)

1. The cycle $C(T_2, e)$ is not contained in the acyclic T_1, so it has an edge $f \notin T_1$; $f \neq e$, and $f \in T_2$, so $f \in T_2 \setminus T_1$.
2. f is in the symmetric difference and $f \neq e$, so $w_f > w_e$, the weights being distinct.
3. $T_2 \cup \{e\} \setminus \{f\}$ is a spanning tree (Lemma 2) of weight

$$w(T_2) + w_e - w_f < w(T_2)$$

a contradiction.

□

Consequences

- With distinct weights, Kruskal's algorithm, Prim's algorithm from any start vertex and every execution of the generic greedy algorithm return the same tree
- On the reference graph, Prim's algorithm from $s = 6$ adds

$$\{5, 6\}, \{4, 5\}, \{3, 4\}, \{1, 3\}, \{2, 3\}$$

once more the tree $\tilde{F}$

When some weights are equal, several minimum spanning trees may exist.

The fundamental cycles and cuts of one optimal tree tell us which.

Proposition 1: edges in every or in no minimum spanning tree

Let T be a minimum spanning tree of $G = (V, E)$.

(a) *An edge $e \in T$ belongs to every minimum spanning tree if and only if $w_e < w_f$ for every $f \neq e$ in the fundamental cut of e.*

(b) *An edge $e \notin T$ belongs to no minimum spanning tree if and only if $w_e > w_f$ for every $f \neq e$ in $C(T, e)$.*

In particular, T is the unique minimum spanning tree if and only if every edge outside T is strictly heavier than the other edges of its fundamental cycle.

Proof (1/3)

(a)

- If the inequalities hold, e is in every minimum spanning tree by the cut property (Theorem 1(b))
- Otherwise some $f \neq e$ in the fundamental cut has $w_f \leq w_e$, and $w_f \geq w_e$ by the cut condition (Theorem 3): so $w_f = w_e$
- $T \cup \{f\} \setminus \{e\}$ is a spanning tree (Lemma 2(2)) of the same weight, without e

Proof (2/3)

(b)

- If the inequalities hold, e is in no minimum spanning tree by the cycle property (Theorem 2(b))
- Otherwise some $f \neq e$ in $C(T, e)$ has $w_f = w_e$, by the cycle condition
- $T \cup \{e\} \setminus \{f\}$ is a minimum spanning tree containing e

Proof (3/3)

For the last claim:

- T is unique if and only if no edge outside T belongs to a minimum spanning tree
- that is, (b) holds for every $e \notin T$

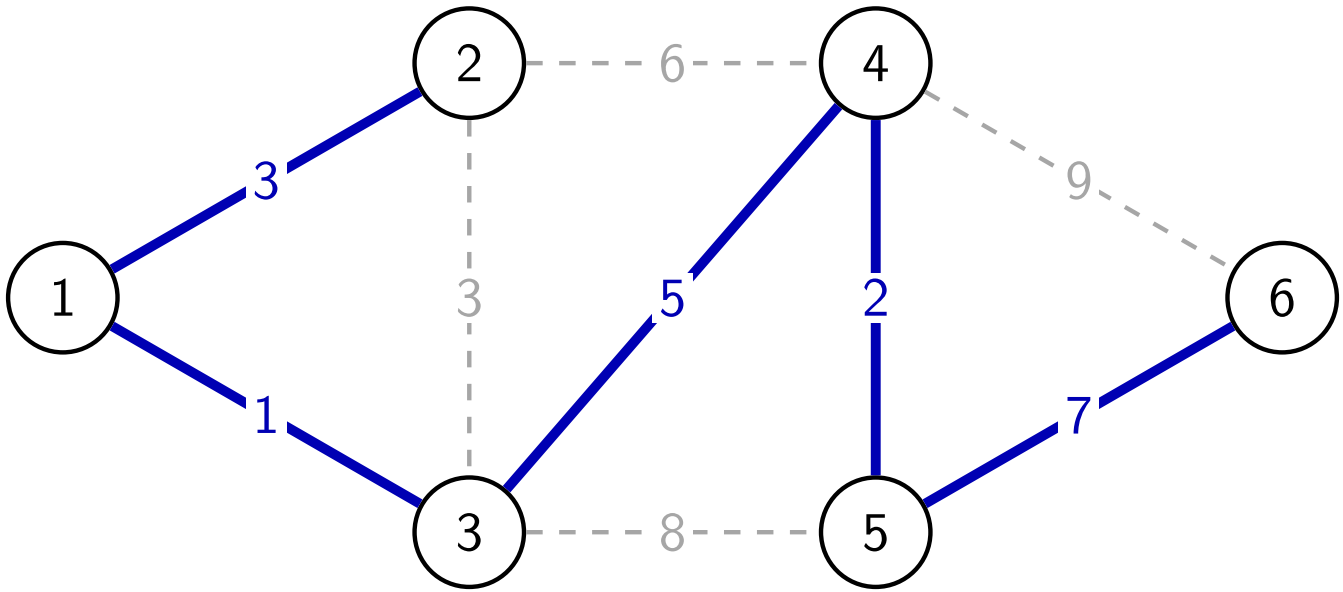
□

Example: ties on the reference graph (1/4)

- Let $w_{12} = 3$ instead of 4, so that $w_{12} = w_{23} = 3$
- The fundamental cycle of $\{1, 2\}$ in $\tilde{F}$ is $1, 3, 2, 1$, with $w_{23} = w_{12}$
- By Proposition 1(b), the tree $\{1, 2\}, \{1, 3\}, \{3, 4\}, \{4, 5\}, \{5, 6\}$ is also optimal, with weight 18

Example: ties on the reference graph (2/4)

With $w_{12} = 3$, the second minimum spanning tree $\{1,2\}, \{1,3\}, \{3,4\}, \{4,5\}, \{5,6\}$ (blue), of weight 18:



Example: ties on the reference graph (3/4)

- Enumeration confirms that these are the only two minimum spanning trees
- Kruskal's algorithm, scanning $\{1, 2\}$ before $\{2, 3\}$, returns the one with $\{1, 2\}$

Example: ties on the reference graph (4/4)

- Ties do not always create several optimal trees
- With $w_{35} = w_{46} = 9$ the tree $\tilde{F}$ is still the unique minimum spanning tree
- Each of the four edges outside it remains strictly the heaviest of its cycle

Uniqueness in one sentence

With distinct weights the minimum spanning tree is unique.

With ties, a tree is the unique optimum if and only if every edge outside it is strictly the heaviest of its fundamental cycle.

Summary

Distinct weights:

- The minimum spanning tree is unique: all the algorithms return the same tree

Weights with ties:

- A tree edge is in every optimum iff it is strictly the lightest of its fundamental cut
- An edge outside the tree is in no optimum iff it is strictly the heaviest of its fundamental cycle
- Equalities in the fundamental cycles give the other optimal trees

Typical mistakes

Introduction: Typical mistakes

- Six mistakes that come up often when the results and the algorithms of this chapter are applied
- They concern the cut property, the tests of Kruskal and of Prim, uniqueness and directed graphs
- Checking them is a quick way to avoid losing points

Observation 1: typical mistakes

- ***Using the cut property with a cut crossed by F.*** *The lightest edge of $\delta(S)$ is safe only if no edge already chosen crosses $\delta(S)$.*
- ***Accepting an edge in Kruskal's algorithm because it touches a new vertex.*** *The test is whether the two endpoints are in different components: $\{3, 4\}$ is accepted although both 3 and 4 are already covered.*
- ***Updating the labels in Prim's algorithm with path lengths.*** *The new label is the weight w_{vu} of a single edge, not $d(v) + w_{vu}$: that is Dijkstra's algorithm for shortest paths.*

Observation: typical mistakes (continued)

- ***Forgetting that the labels can only decrease.*** *In Prim's algorithm $d(u)$ changes only when $w_{vu} < d(u)$.*
- ***Claiming uniqueness from the algorithm.*** *Kruskal and Prim return one tree; with ties other optimal trees may exist, and they are found through the equalities in the fundamental cycles.*
- ***Applying the algorithms to directed graphs.*** *A minimum spanning tree is defined on undirected graphs; in a directed graph the orientation of the arcs is ignored.*

Summary: Typical mistakes

- The cut property needs a cut not crossed by F
- Kruskal tests components, not new vertices
- Prim's labels are single edge weights, and they only decrease
- One tree returned does not mean a unique optimum
- Minimum spanning trees live on undirected graphs

Summary (1/3)

- A minimum spanning tree is a spanning tree of minimum total weight of a connected weighted graph; it has $n - 1$ edges.
- Cut property: the lightest edge of a cut not crossed by F extends F within some minimum spanning tree.
- Cycle property: the heaviest edge of a cycle can be left out, and if it is strictly the heaviest it is in no optimum.

Summary (2/3)

- A spanning tree is optimal if and only if every edge outside it is the heaviest of its fundamental cycle, or equivalently every tree edge is the lightest of its fundamental cut.
- Kruskal's algorithm sorts the edges and accepts those joining two components, in $O(m \log n)$ time; Prim's algorithm grows one tree with the labels $d(v)$, in $O(n^2)$ time.
- With distinct weights the minimum spanning tree is unique.

Summary (3/3)

The weighted reference graph at a glance.

vertices, edges, spanning trees	$n = 6$, $m = 9$, 55
minimum spanning tree	$\{1,3\}, \{2,3\}, \{3,4\}, \{4,5\}, \{5,6\}$, weight 18 (unique)
Kruskal	accepts $\{1,3\}, \{4,5\}, \{2,3\}, \{3,4\}, \{5,6\}$; rejects $\{1,2\}, \{2,4\}$
Prim from 1	selects $3, 2, 4, 5, 6$